- Another Django Admin customization alternative is django-admin-extra-buttons (https://mng.bz/rV1E), which provides tools for adding buttons and wizards to the Django Admin. These can be used to create specialized filters, generate links to outside pages, or wrap views that perform bulk operations.
- *Social auth and account management*—Authentication is a key part of a multi-user website, and even though all of chapter 6 was dedicated to it, there are still plenty of other options out there. Django allows you to build custom user models and modify the authentication backend. A common customization is to replace usernames with email addresses as the login token. Details are available in the documentation: https://mng.bz/Vx2r.
- Of course, why would you build it yourself when others have done it for you? The django-allauth (https://github.com/pennersr/django-allauth) library is a comprehensive approach to authentication. It provides logins based on usernames, email addresses, and social media accounts. It supports Open ID Connect and Oauth 1.0/2.0, meaning over 100 different account providers can be used. It also supports SAML, allowing integration with single sign-on systems as well as two-factor authentication. Even if all that is overkill, it provides a password-management workflow, as covered in chapter 6. Yep, You could have skipped the chapter and simply installed django-allauth instead.

14.5 Asynchronous actions

The basic workflow in Django is synchronous: the user enters a URL, control goes to Django and your views, and the user waits until a response gets returned. A synchronous process covers most of your needs on the web but not all of them. There are two varieties of asynchronous workflows. The first is out-of-band activities, like scheduled jobs. The simplest approach to this is to use a Django management command and your operating system's scheduling tools, like `cron`. For more complex systems, you might need something deeper. The other kind of asynchronous activity is that which interacts with the user. Consider a live-chat session on a web page: the server needs to send information down to the user without it being requested. Django has slowly been adding features to enable this, but even before it did, third-party libraries were capable of handling this scenario:

- *Celery*—A distributed task queue is a system that runs independent jobs across multiple machines. One of the most popular is Celery (https://docs.celeryq.dev/). A Django project that needs an asynchronous task uses a view to kick off a Celery job. Views then periodically query Celery to see if the job is complete. This integration is so common that Celery ships with hooks to work with Django. There is also an extension called django-celery-beat (https://django-celery-beat.readthedocs.io/) that allows you to manage your Celery jobs through the Django Admin. For a good overview on how to get started with Celery and Django, see the following article: https://mng.bz/x26q.